

# Hackathon FIAP X — Fase 5 — Entrega no portal do aluno (PDF)

**Grupo:** Oficina Turbo (106)

**Aluno:** Felipe Ricarte Magalhães

## 1. Dois repositórios de microsserviços (Fase 5 — FIAP X)

| # | Repositório                                                                                                                     | Responsabilidade                                                     | Porta |
|---|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------|
| 1 | <a href="https://github.com/ricartefelipe/fiapx-api-service">https://github.com/ricartefelipe/fiapx-api-service</a>             | API REST — autenticação, upload, listagem de status, download do ZIP | 8080  |
| 2 | <a href="https://github.com/ricartefelipe/fiapx-processor-service">https://github.com/ricartefelipe/fiapx-processor-service</a> | Worker assíncrono — extração de frames com FFmpeg e geração do ZIP   | 8081  |

**Branches:** `main` (estável, CI verde), `develop` (integração).

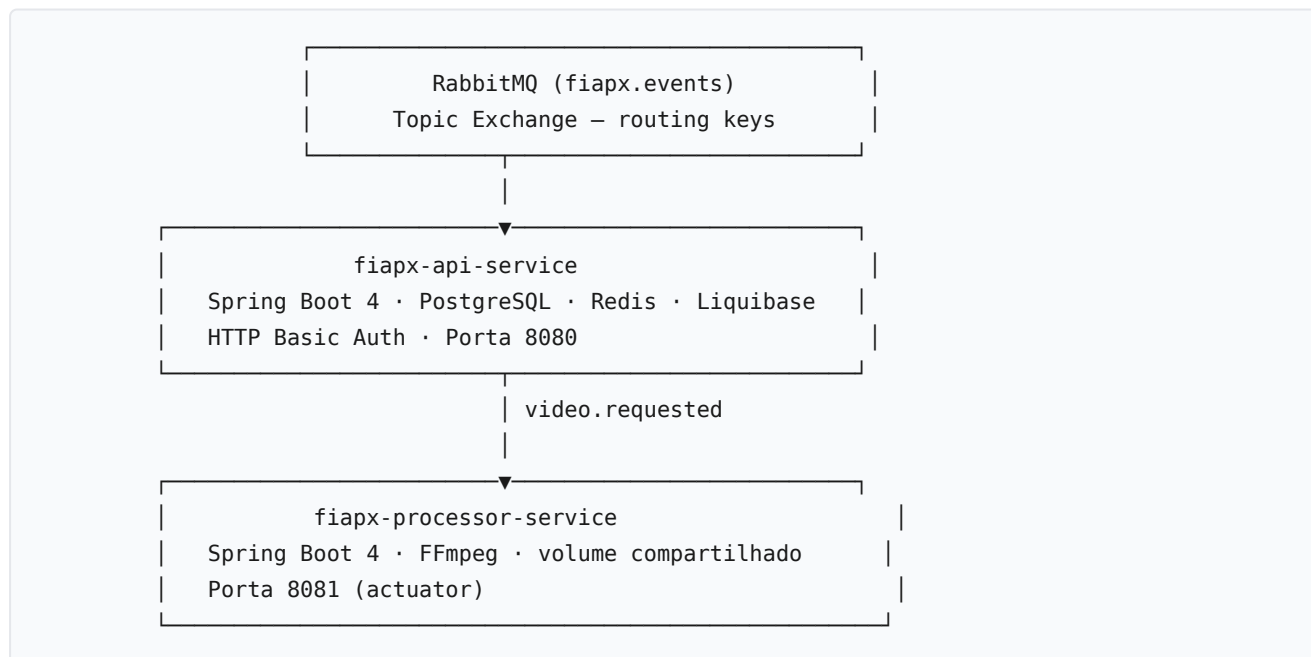
**Acesso ao avaliador SOAT:** usuário `soat-architecture` com leitura nos dois repositórios.

**Documentação técnica no repositório central:**

<https://github.com/ricartefelipe/fiapx-api-service/tree/main/docs/fase5>

## 2. Arquitetura — Microsserviços e fluxo de processamento

### 2.1 Visão geral



**Infraestrutura compartilhada (Docker Compose):** PostgreSQL, RabbitMQ, Redis, Prometheus e Grafana sobem junto com os dois microserviços.

## 2.2 Fluxo upload → fila → FFmpeg → ZIP → download

| Etapa | Componente        | Ação                                                                                  |
|-------|-------------------|---------------------------------------------------------------------------------------|
| 1     | Cliente           | POST /api/videos (multipart) com HTTP Basic Auth                                      |
| 2     | API Service       | Persiste VideoJob (status QUEUED ), salva arquivo e publica video.requested           |
| 3     | RabbitMQ          | Entrega mensagem na fila video.processing                                             |
| 4     | Processor Service | Publica video.processing , extrai frames com FFmpeg, gera ZIP no volume compartilhado |
| 5     | Processor Service | Publica video.completed ou video.failed                                               |
| 6     | API Service       | Atualiza status do job; em erro, notifica via log e e-mail (MailHog)                  |
| 7     | Cliente           | GET /api/videos e GET /api/videos/{id} ; download do ZIP quando COMPLETED             |

## 2.3 Eventos RabbitMQ

| Exchange     | Routing Key      | Produtor  | Consumidor |
|--------------|------------------|-----------|------------|
| fiapx.events | video.requested  | API       | Processor  |
| fiapx.events | video.processing | Processor | API        |
| fiapx.events | video.completed  | Processor | API        |
| fiapx.events | video.failed     | Processor | API        |

## 3. Stack técnica

| Componente       | Tecnologia                                     |
|------------------|------------------------------------------------|
| Linguagem        | Java 21                                        |
| Framework        | Spring Boot 4.0.6                              |
| API REST         | Spring Web + Spring Security (HTTP Basic Auth) |
| Mensageria       | RabbitMQ + Spring AMQP                         |
| Banco relacional | PostgreSQL + Liquibase                         |

|                        |                                                 |
|------------------------|-------------------------------------------------|
| Cache                  | Redis (listagem de jobs por usuário, TTL 5 min) |
| Processamento de vídeo | FFmpeg (via ProcessBuilder no processor)        |
| Documentação API       | SpringDoc OpenAPI / Swagger UI                  |
| Testes                 | JUnit 5 + Mockito                               |
| Cobertura              | JaCoCo                                          |
| CI/CD                  | GitHub Actions                                  |
| Containers             | Docker + Docker Compose                         |
| Observabilidade        | Micrometer + Prometheus + Grafana               |

## 4. Requisitos do hackathon — checklist

Matriz detalhada (requisito × status × evidência × gap): [docs/fase5/matrizes-conformidade.md](https://docs.fase5.com.br/matrizes/matrizes-conformidade)

| Requisito                                  | Status         | Evidência / gap                                                                                                 |
|--------------------------------------------|----------------|-----------------------------------------------------------------------------------------------------------------|
| Processar mais de um vídeo simultaneamente | <b>Parcial</b> | Fila + worker assíncrono configurados; teste paralelo só em <code>scripts/verify-conformidade.sh</code> (local) |
| Não perder requisições em picos de carga   | Conforme       | Filas duráveis; API retorna 202 Accepted                                                                        |
| Proteção por usuário e senha               | Conforme       | HTTP Basic Auth ( <code>fiapx</code> / <code>fiapx123</code> )                                                  |
| Listagem de status dos vídeos por usuário  | Conforme       | <code>GET /api/videos</code> e <code>GET /api/videos/{id}</code>                                                |
| Notificação em caso de erro                | <b>Parcial</b> | Código + MailHog; <b>GitHub main ainda com <code>spring.mail</code> quebrado</b> até push                       |
| Persistência de dados                      | Conforme       | PostgreSQL ( <code>users</code> , <code>video_jobs</code> )                                                     |
| Arquitetura escalável                      | Conforme       | API e processor desacoplados via fila                                                                           |
| Versionamento no GitHub                    | Conforme       | Dois repositórios com GitFlow                                                                                   |
| Testes automatizados                       | <b>Parcial</b> | <code>./mvnw -Pci clean verify</code> ; JaCoCo ~54%; sem Testcontainers/RabbitMQ real                           |
| CI/CD                                      | <b>Parcial</b> | CI (build, testes, imagem GHCR); <b>sem pipeline de deploy</b>                                                  |
| Docker Compose funcional                   | Conforme       | <code>docker compose up -d --build</code>                                                                       |
| Teste E2E (fluxo feliz)                    | Conforme       | <code>./scripts/e2e-test.sh</code>                                                                              |

|                                                        |                 |                                                                               |
|--------------------------------------------------------|-----------------|-------------------------------------------------------------------------------|
| Verificação ampliada (paralelo, falha, mail, métricas) | <b>Parcial</b>  | <code>./scripts/verify-conformidade.sh</code> — local; não no GitHub até push |
| Vídeo de apresentação (≤ 10 min)                       | <b>Pendente</b> | Obrigatório no PDF; ainda não gravado/enviado                                 |
| Evolução do projeto base FIAP                          | <b>Parcial</b>  | Reimplementação equivalente, não fork do repo base                            |

## 5. Cobertura de testes e CI

### 5.1 JaCoCo (local, perfil `ci` )

| Serviço                 | Cobertura de instruções | Testes                                                                                            |
|-------------------------|-------------------------|---------------------------------------------------------------------------------------------------|
| fiapx-api-service       | <b>53,6%</b>            | Unitários (controller, service, security) + <code>@SpringBootTest</code> com H2 — <b>8 testes</b> |
| fiapx-processor-service | <b>56,2%</b>            | Unitários (listener, ZIP, processing) + <code>@SpringBootTest</code> — <b>4 testes</b>            |

Cobertura medida com `./mvnw -Pci clean verify`. Testes usam H2 e mocks (sem Testcontainers). Camadas de integração (RabbitMQ listeners, FFmpeg real) concentram código ainda não coberto por testes unitários.

### 5.2 CI/CD — GitHub Actions

| Repositório             | Workflow                                               | Status em main |
|-------------------------|--------------------------------------------------------|----------------|
| fiapx-api-service       | <code>ci.yml</code> + <code>enforce-gitflow.yml</code> | ✅ Verde        |
| fiapx-processor-service | <code>ci.yml</code> + <code>enforce-gitflow.yml</code> | ✅ Verde        |

#### Evidências:

- <https://github.com/ricartefelipe/fiapx-api-service/actions>
- <https://github.com/ricartefelipe/fiapx-processor-service/actions>

**Imagens Docker publicadas em:** `ghcr.io/ricartefelipe/fiapx-api-service` e `ghcr.io/ricartefelipe/fiapx-processor-service`

### 5.3 SonarCloud

| Item                                 | Status                                                                                                              |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Propriedades no <code>pom.xml</code> | ✅ Configurado ( <code>ricartefelipe_fiapx-api-service</code> , <code>ricartefelipe_fiapx-processor-service</code> ) |

|                          |                                                        |
|--------------------------|--------------------------------------------------------|
| Análise automática no CI | N/A — token SonarCloud ainda não integrado ao pipeline |
| Dashboards               | N/A — aguardando primeira análise publicada            |

## 6. Como executar

### 6.1 Pré-requisitos

- Docker e Docker Compose
- Repositórios `fiapx-api-service` e `fiapx-processor-service` no mesmo diretório pai
- Opcional: FFmpeg no host (para o script E2E gerar vídeo de teste)

### 6.2 Stack completa (recomendado)

```
cd fiapx-api-service
docker compose up -d --build
```

| Serviço                   | URL / Porta                                                                                                     |
|---------------------------|-----------------------------------------------------------------------------------------------------------------|
| API                       | <a href="http://localhost:8080/api">http://localhost:8080/api</a>                                               |
| Swagger UI                | <a href="http://localhost:8080/api/swagger-ui.html">http://localhost:8080/api/swagger-ui.html</a>               |
| Processor (health)        | <a href="http://localhost:8081/actuator/health">http://localhost:8081/actuator/health</a>                       |
| RabbitMQ Management       | <a href="http://localhost:15672">http://localhost:15672</a> (guest/guest)                                       |
| Prometheus                | <a href="http://localhost:9090">http://localhost:9090</a>                                                       |
| Grafana                   | <a href="http://localhost:3000">http://localhost:3000</a> (admin/admin) — dashboard <b>FIAP X — Visão geral</b> |
| MailHog (e-mail de falha) | <a href="http://localhost:8025">http://localhost:8025</a>                                                       |

**Credenciais da API:** `fiapx` / `fiapx123`

### 6.3 Teste E2E automatizado (fluxo feliz)

```
cd fiapx-api-service
chmod +x scripts/e2e-test.sh
./scripts/e2e-test.sh
```

O script aguarda a API, envia upload, poll até `COMPLETED` e valida o download do ZIP.

## 6.4 Verificação de conformidade ampliada

```
cd fiapx-api-service
chmod +x scripts/verify-conformidade.sh
./scripts/verify-conformidade.sh
```

Valida: health da API (8080) e processor (8081), dois uploads paralelos, falha com arquivo inválido + e-mail no MailHog, targets Prometheus UP.

## 6.5 Testes unitários

```
./mvnw -Pci clean verify # em cada repositório
```

## 7. GitFlow aplicado

Fluxo documentado em `docs/GITFLOW.md` (ambos os repositórios):

| Branch                 | Uso                                                                      |
|------------------------|--------------------------------------------------------------------------|
| <code>feature/*</code> | Novas funcionalidades → PR para <code>develop</code>                     |
| <code>fix/*</code>     | Correções → PR para <code>develop</code>                                 |
| <code>release/*</code> | Preparar versão → PR <code>develop</code> → <code>main</code>            |
| <code>hotfix/*</code>  | Correção urgente → PR para <code>main</code> + sync <code>develop</code> |

O workflow `enforce-gitflow.yml` **bloqueia** PR de `feature/*` direto para `main`.

**Release entregue:** PR #4 ( `develop` → `main` ) mergeado com CI verde em 24/07/2026.

## 8. Observabilidade

| Ferramenta | Acesso                                                             |
|------------|--------------------------------------------------------------------|
| Prometheus | <a href="http://localhost:9090">http://localhost:9090</a>          |
| Grafana    | <a href="http://localhost:3000">http://localhost:3000</a>          |
| Métricas   | <code>/actuator/prometheus</code> em API (8080) e Processor (8081) |

## 9. Vídeo demonstrativo (≤ 10 minutos, sem áudio)

**Arquivo local:** docs/delivery/fiapx-fase5-demo.mp4 (~4 min 46 s, mudo)

**Como regenerar:**

```
docker compose up -d --build
python3 scripts/capture-demo-frames.py
SECONDS_PER_FRAME=22 ./scripts/generate-demo-video.sh
```

Conteúdo: arquitetura, capturas de Swagger/RabbitMQ/MailHog/Grafana/Prometheus, upload, processamento, erro + e-mail, testes e GitHub.

**Portal:** enviar o MP4 ou publicar no YouTube/Vimeo e colar o link no campo do aluno.

---

## 10. Pacote zipado (portal)

Arquivo **fase5-microservicos.zip** enviado no portal contém:

- Código-fonte de **fiapx-api-service** (sem **target/**, sem **.git/**)
- Código-fonte de **fiapx-processor-service** (sem **target/**, sem **.git/**)
- Este PDF ( entrega-portal-fase5.pdf )

Arquivo **fase5.zip** contém a pasta **fase5/** com o PDF e o ZIP de código.

---

*Entrega Fase 5 — Hackathon FIAP X — plataforma de processamento de vídeos em microserviços com RabbitMQ, PostgreSQL e Docker Compose.*